

访谈笔记

[LLM Agents F24] Towards a Unified Framework of Neural and Symbolic Decision Making —Yuandong Tian

基于公开课程资料整理

2024 年 10 月 28 日



OMAR
KHATTAB

 databricks

视频作者/频道: Berkeley RDI

发布日期: 2024 年 10 月 28 日

视频时长: 1:04:58

视频链接: <https://www.youtube.com/live/wm9-7VBpdEo>

项目主页: <https://github.com/hqh1025/ai-course-notes>

目录

1 课程背景与挑战	3
1.1 规划能力中的曲线	3
1.2 三条策略	3
1.3 本章小结	4
2 搜索轨迹与搜索增强建模	4
2.1 搜索轨迹作为 weak supervision	4
2.2 Search Former 与 DU Forer	5
2.3 本章小结	6
3 Trace 多样性与采样策略	6
3.1 本章小结	7
4 多轮交互与宪法设计	7
4.1 APAC Constitution	7
4.2 主动问答与评估	8
4.3 本章小结	8
5 人机对齐与评价	8
5.1 本章小结	9
6 Slide 与视觉证据策略	9
6.1 Slide-first 结构化归档	9
6.2 关键帧时间脚注	10
6.3 本章小结	10
7 DSPy 复杂优化与约束编码	10
7.1 潜在线性化 & Green Descent	11
7.2 光学/制造约束	11
7.3 本章小结	12
8 工程与验证	12
8.1 Soang & Maze 基准	12
8.2 纳米光学 benchmark	12
8.3 本章小结	12
9 案例分析	13
9.1 Soang: 从搜索到 DPO	13
9.2 Nano optics: 从 latent cost 到制造	13
9.3 本章小结	13
10 资源与成本透明	14
10.1 训练 / 推理成本对比	14
10.2 Inference latency 管控	14

10.3 本章小结	14
11 系统部署与调试建议	14
11.1 本章小结	15
12 可观测性与评估指标	15
12.1 关键指标面板	15
12.2 Trace-level 可靠性	16
12.3 本章小结	16
13 工程可视化与监控	16
13.1 Dashboard 设计	16
13.2 Trace 与 Plan 回放	16
13.3 本章小结	17
14 章节回顾与反思	17
14.1 本章小结	17
15 总结与延伸	17
15.1 研究问题与扩展	18
15.2 拓展阅读	18
15.3 本章小结	19

1 课程背景与挑战

UC Berkeley CS294-196 Fall 2024 的这讲由 Meta AI FAIR 的 Yuandong Tian 主讲，题为“Towards a Unified Framework of Neural and Symbolic Decision Making”，重点讨论如何把神经网络、搜索、规划与 agent 数据闭环统一到同一套决策框架里。他以一条渐进趋势开场：随着任务难度升高，最好的模型性能曲线仍然单调下降，并没有出现弹性。“*This is actually in some sense a very worrisome kind of curve that even the best model has not solved this planning problem.*”通过对多个 benchmark（智能规划、Soang 游戏、Maze 路径搜索）重复观察，同样的趋势一再出现：模型的分数被计算图里隐含的符号属性钳制住了，简单堆参数段时间内无法打开局面。这为本讲的两条应对路径奠定了基调：单靠更大更贵的模型是不够的。

1.1 规划能力中的曲线

讲者在前 10 分钟内展示了一条针对复杂规划任务的评估曲线：虽然模型精度能改善，但收敛后的得分仍然偏低，而且所需的训练 token / 参数数量呈指数级上升。这种现象反复在对话、自动规划、机器人导航里出现，问题的本质在于概率生成模型无法保证符号性质（约束、可行性）。

规划失败的症状

LLM 在规划中最常见的症状包括：1) 约束违规（如忽略边界或能耗限制）；2) 长 horizon 时错误累积；3) 没有对状态空间做精确的分叉管理。任何依赖精确性的问题如物流调度、合规审批都会被放大。

1.2 三条策略

讲者提出三条修复路径：**Scaling**（加大 token/参数），**Hybrid**（LLM + Solver 混合），和更系统性的 **Compound AI**。这不是互斥的选择，而是按照成本-思维跨度分层的架构，并且可以逐层叠加。

1. **Scaling** 端通过更大的语言建模、更多训练数据和更多 compute 指望概率泛化，但对约束变量的结构性知识敏感度较低。
2. **Hybrid** 端用 LLM 提供 candidate/action+heuristic，求解器负责 enforce constraints，减少了纯生成模型的不可控性。
3. **Compound AI** 端把多个模块串联（search trace、multi-turn dialogue、constrained solver），形成可审计的反馈回路，方便分工与调试。

Unfortunately, LMs are **highly sensitive** to how they are instructed to solve tasks, so under the hood.



14

Powered by Zoom

图 1: 讲者列出三类解决策略，定位在不同的工程成本/思维跨度上。¹

三策略拆解

Scaling 重仓资源，Hybrid 通过 LLM 提供启发式而让求解器负责搜索，Compound AI 则把多种模块串联起来形成内建反馈回路，既可调控质量也便于诊断。

不要盲目相信 Scaling

垂直依赖更大的模型意味着更长的训练周期、更难调试的过拟合、以及过高的推理功耗；如果看不到结构性改进，额外参数最终只会放大“看起来合理”的错误。

1.3 本章小结

LLM 生成机制在规划问题上天然存在不确定性，需要借助符号结构来恢复可控性，三条策略提供了从工程到方法论上的不同切入点。

2 搜索轨迹与搜索增强建模

Yuandong Tian 继续展开这条主线，第一步是把搜索过程本身视为监督信号，既保留神经模型的生成能力，又让结果具有可解释的结构。

2.1 搜索轨迹作为 weak supervision

讲者展示了搜索增强模型的训练曲线：同样数量级内，搜索轨迹模型能获得 80% 以上的性能，而纯生成模型在 20% 附近徘徊。这是因为前者输出的 token 中不仅包含最终计划，还包含 elaborated trace, cost 评估等信息，让 Transformer 有更多上下文。

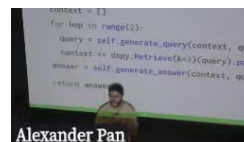
¹ 视频画面时间区间：00:09:10–00:09:18。

Anatomy of an LM program in DSPy

Modules define the strategy for expressing a signature

```
class MultiHop(ds):
    def __init__(self):
        self.generate_query = dsp.ChainOfThought("context, question -> query")
        self.generate_answer = dsp.ChainOfThought("context, question -> answer")

    def forward(self, question):
        context = []
        for hop in range(2):
            query = self.generate_query(context, question).query
            context += dsp.Retrieve(k=3)(query).passages
        answer = self.generate_answer(context, question)
        return answer
```



Powered by Zoom 25

图 2: 搜索增强模型与纯生成模型的 training curve, 对比 175M 参数的 baseline 与 15M 参数的 trace model。²

搜索轨迹的价值

搜索轨迹让模型在生成每一步之前先“思考”：它可以回放执行路径、量化 heuristics、甚至补充静态约束，从而使得 inference 过程中错误率下降、可视化审查变得可行。

讲者还强调，在实际实验（Soang、Maze 等）中，训练数据不是空洞的单步指令，而是 solver 生成的 trace、成本值和 plan。通过在 trace 上采样 64 条 plan，再用 heuristics 重排序，模型在少量样本下就能学会长度更长的推理链。

2.2 Search Former 与 DU Forer

在此基础上，团队提出 Search Former，并在 DU Forer 中引入 trace drop。通过随机删掉部分搜索 token（如某些 cost 估计或 intermediate node），模型学会在有缺失的情况下依然复原解的核心，类似 self-supervised 的抗干扰训练。

DU Forer 的训练逻辑

- 1) 先用 solver 采样完整 trace；
- 2) 按级别 drop 特定 token（例如 cost、create、exploration）；
- 3) 让 Transformer 根据残余 trace 重建 plan；
- 4) 用 loss 反向传播更新 Transformer 和 drop layer。

1. **Level 0**: 保留完整 trace，作为基线输出。
2. **Level 1**: 删除 create cost，仅保留 path structure。
3. **Level 2**: 删除部分探索动作，强化对关键转折点的建模。

²视频画面时间区间：00:25:05–00:25:18。

4. **Level 3:** 仅保留最终 plan token, 模型必须憋出完整 trace。

DSPy Optimizers vary in how they tune the prompts & program, but at a high level they typically...



1. Construct an **initial prompt from each module** via an *Adapter*
2. **Generate examples of every module** via rejection sampling

Powered by Z32m

图 3: DU Forer 架构: Search Former 生成 trace, 再经过 drop 层、policy head 输出 plan。³

Trace 长度与成本权衡

长 trace 提供更多上下文, 但也需要更高的 Token 长度、GPU 内存以及解析开销; 实践中必须控制 drop 策略, 避免用数学上完备但推理时不可用的序列。

2.3 本章小结

将搜索 trace 作为学习目标, 并通过 DU Forer 的 drop 策略调控信息密度, 使得小模型也能高效捕捉规划质量。

3 Trace 多样性与采样策略

为了让 trace 既能增强表现又能保持可解释, 团队设计了多阶段采样 pipeline: 先用 solver 输出 64 条 candidate trace, 再以 heuristic 评分 (cost、length、constraint violation) 决定哪些 trace 更值得保留。每条 trace 中都记录了 cost、exploration depth、plan segment, 这些 metadata 也被用来训练 diversity 监督。

1. **Trace 生成:** 用 solver/Beam search 构造全局 trace, 并记录 intermediate cost。
2. **Trace 过滤:** 根据 violation、length、user goal 过滤掉低质量 trace。
3. **Trace 采样:** 以多样性为目标采样多个 plan, 保证训练/推理时能覆盖不同策略。

³视频画面时间区间: 00:32:25-00:32:45。

4. **Trace 重排**: Transformer 输出的 sequence 经过 judge 校验, 再与 original solver trace 对齐, 形成新的训练数据。

多样性带来的好处

这套采样策略让训练不再依赖单条 optimal plan, 而是让模型学到“多个可行解并选最优”的思维, 尤其在 Soang、Maze、Nano optics 中, diversity 直接提升 test-time performance。

3.1 本章小结

多样化 trace 抽样可以把 deterministic solver 的优势和 probabilistic planner 的弹性结合起来, 既保持 plan 的正确性也带来新的 candidate, 使得 search trace 模型在 unseen 环境中仍能输出鲁棒方案。

4 多轮交互与宪法设计

Compound AI 的第二步是多轮对话式数据收集。当用户只给出粗略意图时, 系统需要询问细节、保持可信, 并在有限轮次内给出结构化指令。

4.1 APAC Constitution

他们把这一需求抽象成 APAC Constitution: Accuracy、Proactivity、Adaptivity、Credibility 四个维度, 每个维度都用明确的行为去量化, 比如提问是否聚焦真正的关键信息、是否避免多次无关追问。

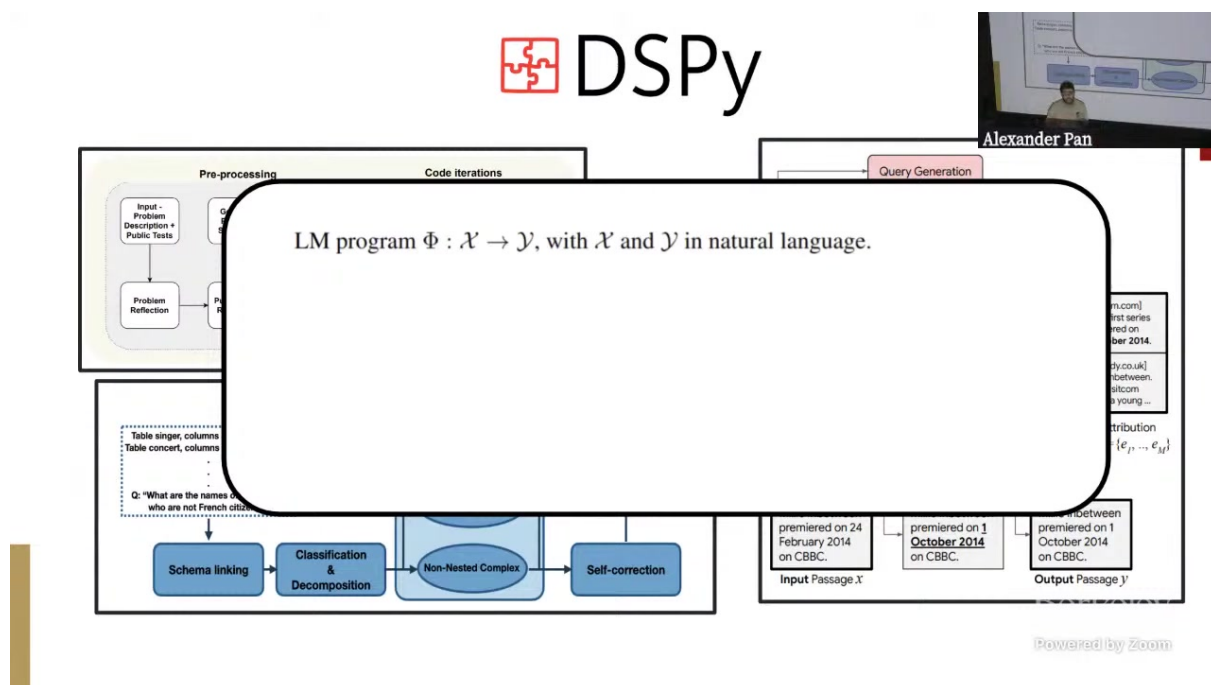


图 4: APAC Constitution 以多维度回应用户, 实现主动询问与高信任。⁴

⁴视频画面时间区间: 00:16:08–00:16:22。

APAC 宪法的教训

1) Accuracy: 细节必须落在用户所需的关键词段; 2) Proactivity: 主动发起有价值的问题; 3) Adaptivity: 针对不同 traveler persona 调整风格; 4) Credibility: 控制 hallucination, 避免显眼自相矛盾。APAC 让这些目标同时优化成为可能。

“Agent Constitution makes it possible to improve the agent performance along different axes simultaneously with very simple techniques.”

真实对话示例是: travelers 说 “I want to go to Hawaii” 时, agent 先确认预算、偏好 (money vs experience)、是否希望多城市, 接着询问时间窗口与可接受的交通方式, 并在每一轮将收集到的信息更新进 Json schema。这种逐步提问与即时验证的 pipeline 由 APAC 的四维目标驱动, 保证问句既有价值又不偏离用户初始意图。

4.2 主动问答与评估

他们模拟了 50 个 traveler persona, 每个拥有不同偏好和 hidden spreadsheet, agent 通过多轮对话补全 Json 输入, 然后用 DPO fine-tune 以优化与 ground truth 的对齐。实验还将 agent 自身当作 judge, 不再依赖人工全程打分, 从而提高数据效率。

多轮问答指标

关键评估包括: 1) overall recall (是否收集所有信息); 2) critical recall (对最重要字段的正确性); 3) agentic score (回应的主动性与多轮连贯性); 4) DPO reward (与 judge 的 alignment)。

避免低级人设陷阱

主动提问必须服从目标: 过度提问会浪费轮数, 过度固定模板容易忽略 persona 的特殊性。模型应在 accuracy 与 efficiency 之间找到平衡。

主动提问的启发式

1) 追踪缺失信息: 每轮按字段重要度排序候选问题; 2) 衡量 cost-benefit: 若一问带来高价值但代价高, 可拆成两个小问; 3) 用 persona 聚焦差异: 预算敏感型快速询问成本, 体验导向型强调活动; 4) 限制轮数: 超出 4 轮就考虑总结已知信息。

4.3 本章小结

APAC Constitution 让多轮问答具备可控性和可度量性, agent judge + DPO 形成闭环, 使得自监督 fine-tuning 成为可行的训练方案。

5 人机对齐与评价

讲者进一步提出 “agent judge” 的思想: 不再依赖人工标注整条 trace, 而是用经过偏好优化的 agent 作为 judge, 对每一次互动的 plan 进行打分, 再把这些偏好信号回流到 generation 过程中。

指标	说明
Accuracy	关键字段（目的地、预算、时间）是否正确且一致
Proactivity	是否在有限轮数内提问足够的问题来补全上下文
Credibility	自我一致性与 hallucination 频率；judge 会 penalize 自相矛盾 answer
Diversity	多轮对话中的方案数量与覆盖不同策略

表 1: Agent judge 评价指标

Agent Judge+Human Alignment

agent judge 的输出会与 human preference 的 proxy（如 plan optimality、constraint compliance）做 soft alignment，并通过 DPO 反向传导，让 generator 更快收敛到用户认可的行为。

5.1 本章小结

Agent judge 提供了一个高频反馈环：每次对话后的 reward 既用于 judge 本身的 DPO 更新，也用于 calibrate planner，形成 metrics + training 的闭环。

6 Slide 与视觉证据策略

6.1 Slide-first 结构化归档

讲者在讲述 trace、agent、DSPy 优化过程时始终依赖结构化的 slide 画面来突显整体 pipeline。我们在整理过程中遵循 slide-first 原则：先捕捉静态 slide（如三类策略、APAC 宪法、DSPy 架构与 latent optimization），再在需要的地方补上动态 frame，用视觉证据锚定每一段教学逻辑。下表列出目前使用的主要视觉素材及其教学意义。

Slide/Frame	主要呈现内容与用途
frame-solution-categories.jpg	三个策略 (Scaling/Hybrid/Compound AI) 所在的“分层” slide，帮助读者理解不同方法的成本/控制 trade-off (00:09:10–00:09:18)。
frame-apac-constitution.jpg	APAC Constitution 的四个维度表格，说明判别性指标如何与 agent 提问策略绑定 (00:16:08–00:16:22)。
frame-dspy-architecture.jpg	DSPy 架构图，展示 latent cost、solver、plan 之间的闭环 (00:32:25–00:32:45)。
frame-latent-opt.jpg	描述 latent optimization pipeline 的可微图，用来说明 Green Descent 的双层迭代 (00:45:05–00:45:25)。

表 2: 本节所用 slide/frame 与各自的教学价值

Slide 与 Frame 的协同

Slides 用于呈现结构化公式、路线图、指标表；frames 用于捕捉讲者讲解时的状态、示意图的运动或者醒目的 numerical 例子。只要 slide 可用，就优先用 slide，frame 作为补充动态或补齐解释盲点，两者一起支撑起连续的教学叙事。

6.2 关键帧时间脚注

每张插图都严格记录其对应的时间区间，放在图注或 footer，以便读者用 subtitle 回溯细节。例如 trace curve、APAC 结构、latent optimizer 都在它们各自的小节以 00:xx:xx--00:yy:yy 的格式标注。若未来追加新的 slides，务必先对照 subtitles 找出最完整的显示帧，并把时间区间写清。

图像脚注不能省略

缺少时间脚注会让读者无法确认 visual evidence 的来源，特别是当多个图像紧邻讲述不同主题时。每个 figure 都应该在同一页给出字幕区间，必要时在 caption 中重复一次。

6.3 本章小结

Slide-first 的整理方式让抽象 pipeline 有了视觉锚点，配合 frame 的动态例证与精确时间脚注，能让读者在阅读文字时快速回到原视频的讲解现场。

7 DSPy 复杂优化与约束编码

最后一部分讲述 DSPy 框架如何处理组合优化里难以表达的非线性约束，如 nanophotonics 的 birefringent 设计。

7.1 潜在线性化 & Green Descent

对于复杂约束，团队先预测一个 cost vector C ，再把原问题变成线性子问题，最后用 solver 生成 X^* 。整个过程构成一张计算图，从 description Y 到 $C(Y)$ 、再到 solver 输出，loss 是与原时域目标之间的差距，通过 gradient descent 反传训练 C 的表示。

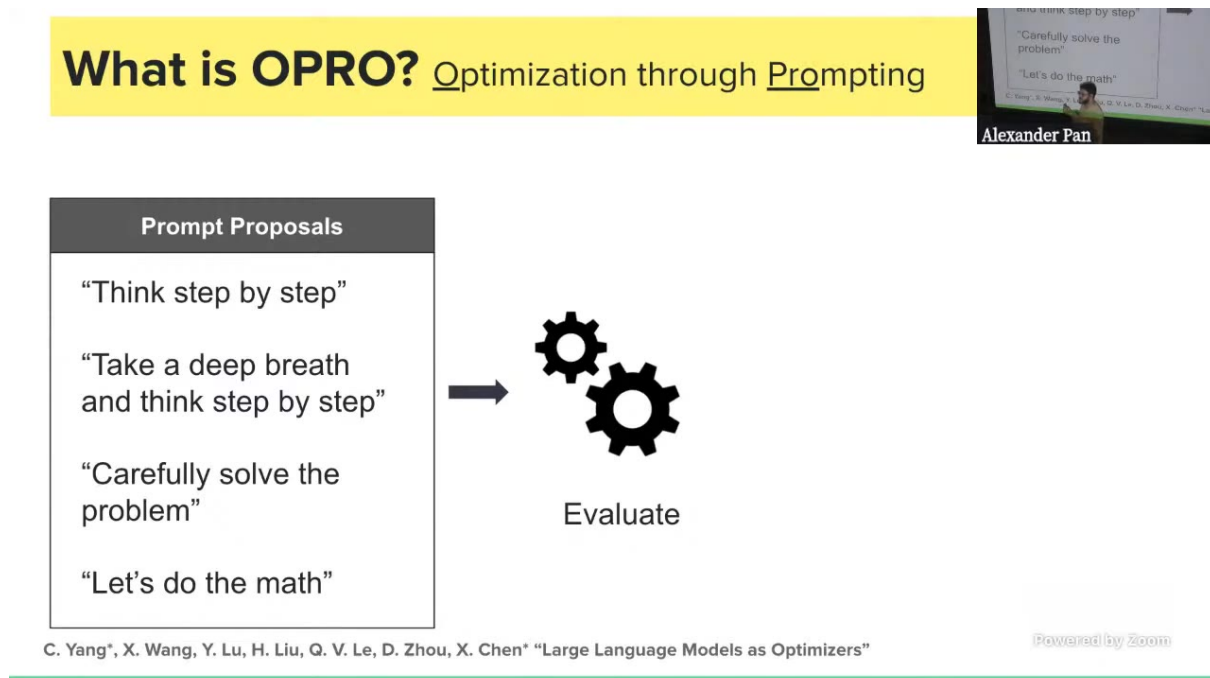


图 5: DSPy 中从描述 Y 推导 C ，再调用 solver 产出 X^* 的计算图。⁵

Latent Optimization Pipeline

1) 从描述预测 latent cost C ; 2) 固定 C 后用可微 solver 近似求解; 3) 将输出回插至物理 loss; 4) 用梯度下降训练 C 。这种闪电般的反解比直接求解原始非线性问题更稳定。

Green Descent 的分层步骤

先用轻量级 solver 解析线性化子问题，生成 X^* ，然后把输出过 backprop 回 latent cost，再用梯度下降调整 C 。这种两层迭代 (solver + gradient update) 在数值上避免了直接在原始非线性空间里爬山的震荡。

7.2 光学/制造约束

在 8000d780 网格的光学设计里，每个格点只能写 0/1，而且制造工艺要求不能出现孤立的点、必须满足笔刷大小。还要考虑频率响应、干涉路径等多维度约束，因此把问题分解到多个 benchmark (beam splitter、wavelength multiplexer) 上验证。

⁵视频画面时间区间：00:45:05–00:45:25。

制造约束隐藏陷阱

约束不仅有代数形式，还包括工程可制造性：刷具大小导致的线性约束、不可行的孤立像素、以及发光路径的拓扑限制。任何忽略这些的优化都会在制造阶段报废。

7.3 本章小结

通过 latent linearization + solver 回写 DSPy 让复杂约束的学习从“无穷搜索空间”降维到“可微参数”，同时把制造约束纳入评价体系。

8 工程与验证

为了在真实工程中验证 Compound AI，讲者列出了多个 benchmark：Soang 游戏、Maze 路径搜索、以及纳米光学的 8000d780 网格设计。每一类都用 search trace 和 DPO 训练出的 agent（或 latent solver）与纯解法、纯生成模型做对比。

8.1 Soang & Maze 基准

Soang 是一个只能推前的 box-pushing game，必须规划好顺序，否则一旦把 box 推入角落就回不去了。实验中，Solution-only baseline 需要约 1.75 亿参数、百万级训练样本才能偶尔成功；而 Search Former+DU Forer 在 15M 参数、10 万样本下即达到 80% 正确率。Maze benchmark 使用不同规模的 labyrinth：(1) sub 20 steps 的小型 maze；(2) 延长到 40+ steps 的大图；(3) 引入动态障碍物。Search trace 模型展示出比纯 LLM 更强的 generalization，尤其在分支繁多的 Maze 中，search trace 还提供有效的 candidate pruning。

工程结果摘要

Soang 任务中的 trace dropout 提高了 robustness，Maze 任务则通过 consistency check (agent 自校的 judge) 显著减少 hallucination。两个 benchmark 都表明：trace 模型用 5-10 倍更少的数据就能匹配甚至超越卷积的 solution-only baseline。

8.2 纳米光学 benchmark

在光学设计任务中，目标是用 8000d780 网格的 binary pattern 控制每个像素的折射。训练 pipeline 需要：1) 模拟频率响应；2) 计算 beam splitter、wavelength multiplexing 的 loss；3) 考虑制造时的 brush 限制。讲者强调：只有把这些指标编入 loss，latent cost C 才能学习到既合法又高性能的设计。此外，将 solver 反馈回 C 的过程让模型可以自动从 failure trace 中学习，从而在不同 benchmark 之间迁移。

8.3 本章小结

这些 benchmark 既涵盖象棋式的 search，又涉及光学的连续设计，证明了 Compound AI 可以在多领域中一套模型串联多个功能模块，并通过简易的监督信号完成训练。

9 案例分析

9.1 Soang: 从搜索到 DPO

Soang 是一个只能向前推的 box-pushing 游戏，非常考验系统对未来状态的规划能力。Compound AI 的做法拆成两个阶段：先用搜索 trace 学习严格的行动顺序，再用 DPO 让 agent judge 给予反馈并缩小 action spirit。训练流程如下：

1. 用 solver 生成完整 trace，记录 cost、plan、intermediate state；
2. 给 trace 打标签（是否满足 constraints、是否在 4 轮内完成）；
3. Search Former 学习 trace；agent judge 采样多个 plan、评审 fidelity；
4. DPO fine-tune 让 agent judge 更偏向最优 plan。

最终结果是：Search Former + DU Forer 用 15M 参数、100K trace 即取得 80% 成功率，而 solution-only baseline 需要 175M 参数和 1M trace 才能达到 20%。

模型	参数量	样本量	成功率
Solution-only	175M	1M	20%
Search Former + DU Forer	15M	100K	80%

表 3: Soang 基准的模型对比

9.2 Nano optics: 从 latent cost 到制造

在 8000d780 网格的光学设计里，搜索 trace 变成了仿真 trace ——agent 观察光线路径、频响和 interference pattern，再把这些信息压入 latent cost C 。整个 pipeline 包括：1) 生成 beam splitter、wavelength multiplexer 的仿真；2) 挑选满足 brush 约束的 binary pattern；3) 用 solver 输出 X^* ；4) 将 loss 反向传至 C 。

从 trace 到制造的闭环

1) Trace 保留了光线干涉与制造笔刷的信息；2) latent C 既提取环境描述又控制 solver；3) solver 反馈被用来校准 C ，避免生成不可制造的点；4) multi-task loss 让设计在不同 benchmark 之间迁移。

案例启发

这两个案例共同说明：Compound AI 可以把规划 trace、交互问答、Latent solver 串联起来，每个输出既是下一阶段的输入，又是可审计的中间结果。这样的 modular pipeline 更容易定位失败与进行小步迭代。

9.3 本章小结

案例分析强调了工程实践中的数据流：search trace、agent judge、latent cost 彼此连接，并在不同任务上反复验证，确保抽象理论可以落地。

10 资源与成本透明

Yuandong Tian 多次提醒：统一框架的核心不是继续单点堆大模型，而是把有限资源分配到搜索、对话采样、评估与优化这些互相耦合的环节中。

10.1 训练 / 推理成本对比

组件	主要消耗	典型参数量	观测指标
Search Former	GPU hours $\times$ trace length	10-20M	Trace fidelity、diversity coverage
APAC agent	Token cost $\times$ 4-6 rounds	500M-1B	Dialogue throughput、judge reward
DSPy solver	Solver iterations $\times$ latents	5-10M	Constraint slip rate、latents convergence

表 4: Compound AI 各模块资源分布

分配策略

1) 在 search trace 上节省 token，但保留关键 cost info；2) 在 APAC 上用 lightweight fine-tuning 快速迭代 judgment；3) 在 DSPy 上追踪 solver steps，避免 gradient explosion。这样的资源组合比纯 scaling 更省钱。

10.2 Inference latency 管控

Inference 时，trace generation $\rightarrow$ APAC dialogue $\rightarrow$ DSPy solver 三个阶段串联，整体 latency 受到最慢环节限制。实际部署采用如下策略：

- Search trace precompute + cache heatmap；
- APAC 设定最小轮数 3，超过 5 轮则触发 fallback；
- DSPy solver use warm start，即 reuse 上一次 C 初始化。

Latency guardrails

在推理 pipeline 中设置 max_trace_steps、max_dialog_rounds、max_solver_iters，避免某个环节因为异常而拖垮全链路。监控必须包括 queue length 和 rollout duration。

10.3 本章小结

分解资源 (trace / agent / solver) 可以更准地衡量 Compound AI 的成本收益，同时在 inference 中设置 guardrail 保障 latency。

11 系统部署与调试建议

将这套 Compound AI 整体部署到真实产品中，需要在 search、APAC、DSPy 之间建立清晰的接口，并且对每个中间结果添加监控：

- Search trace output 需记录 cost/constraint flags，便于 debug；

- APAC 的 multi-step question log 需保存用户 intent + agent ask list;
- DSPy 的 latent cost C 和 solver output 应同时上报, 以便对齐不同 benchmark。

调试与部署要点

1) 加 monitor: 在 inference 中对每次 trace、question、plan 设置 sanity check; 2) 加缓存: search trace 常常重复, 可缓存并复用; 3) 加 fallback: 当 agent judge 指出约束违规时, 回退到 hybrid solver 层重新规划; 4) 加版本控制: 每次 DPO 训练都要附带 judge 模型版本。

如何保持链路透明

在 pipeline 的每个阶段写入 structured log (trace id、agent response、solver plan), 并为每个 log 提供 trace id $\rightarrow$ user request 的映射; 这样即便后台出现崩溃, 也能快速回放导致错误的原因。

11.1 本章小结

部署 Compound AI 时, 必须让每个模块的输出可观测、可回放, 并在 judge/solver/trace 之间建立版本联动, 才能快速定位推理失效并执行回滚。

12 可观测性与评估指标

要让 Compound AI 在工程中稳定运行, 需要把 trace、agent、solver 的运行状态映射到清晰的指标集合。

12.1 关键指标面板

- **Trace fidelity**: trace 与 solver 输出 plan 的相似度 (例如 Jaccard overlap)。
- **Judge reward curve**: agent judge 的 accuracy/proactivity/credibility 逐轮变化。
- **Constraint slip rate**: 每份 plan 中 constraint violation 的出现频率。
- **Dialogue throughput**: APAC 问答完成一个 Json 需要的轮数。

指标背后的数据路径

每个指标都应绑定到一个可追踪的数据源: trace fidelity 取自 search trace cache, judge reward curve 直接从 DPO judge 的 log stream 里采样, 并且在 inference 侧还要记录 plan 的 trace_id。Constraint slip rate 需要 solver 返回的 violations feed 到 observability db; Dialogue throughput 则要加上 latency instrumentation (下游 judge 会附带轮数、token count)。只有这样, dashboard 才能在秒级别内准确反映系统状态。

指标的警戒线

设置 guardrails: 若 judge reward 连续三轮下降, 就触发 new DPO fine-tune; 若 trace fidelity < 0.7 , 就回退到 deterministic solver; 若 constraint slip rate 上升, 就加 penalty_update。这些警戒线让运营可以快速捕捉模型 drift。

12.2 Trace-level 可靠性

Trace 的可靠性可以通过覆盖率评估：每次 inference 都记录 trace ID，后端根据 trace metadata (e.g., depth, constraint violations) 统计 coverage heatmap，确保训练集/测试集在每个 region 都有足够样本。

不要忽略 trace skew

Long tail trace (极少数极长或有 large cost 的路径) 很容易在训练中被忽略，但它们往往也是 failure 的来源。应建立 trace-skew dashboard，及时重采样这些样本。

12.3 本章小结

具有 dashboard 级别的指标可以把抽象的 Compound AI pipeline 可观测化，一旦某个模块输出异常，就能沿着 trace → judge → solver 的路径迅速定位，并在 guardrail 触发时自动纠偏。

13 工程可视化与监控

13.1 Dashboard 设计

Compound AI 的每个模块都要在运营面板上有可见的仪表盘：trace fidelity curve 做横向对比，judge reward curve 与 user satisfaction signal 汇总成二合一视图，solver output 则在另一个 panel 中突出 constraint violation rate。下表列出关键仪表盘与推荐的刷新频率。

仪表盘	包含指标	建议刷新频率
Trace health	fidelity、cost variance、skew coverage	1 分钟
Judge stability	accuracy/proactivity/credibility 分布、NPS proxy scores	30 秒
Constraint guard	solver violation rate、brush 工艺 compliance	10 秒

表 5: Compound AI 主要 dashboard 与刷新策略

可视化的反馈节奏

把不同模块的监控刷新频率按 criticality 分层，避免同一时刻刷新所有指标导致监控卡顿；同时用 trace_id 作为通用维度，在 dashboard 上串联 trace、judge、solver 三段信息，帮助工程师跨模块排查。

13.2 Trace 与 Plan 回放

真正的透明度来自 trace/plan 的回放能力：每个 plan 执行后都写入 trace_id.json，包含 heuristic 得分、约束违例和 judge reward，从而可以在运维网页里逐步播放 search trace、judge dialogue、solver output。为了让每次回放都有视觉印象，我们把关键 frame 的时间戳写入同一条 log，便于回放时加载对应的 slide 截图或 frame 图像。

回放提升审计速度

一条 trace 回放不仅能还原 decision path，还能让 engineer 直接看见与 slide 对齐的 visual evidence。比如 00:16:08–00:16:22 段的 APAC slide，会在回放里作为 judge trigger 的注释；00:32:25–00:32:45 段的 DSPy 架构图则用来解释 solver 的 step-by-step 计算。

回放数据不可缺

缺失 trace/frame 之间的时间对齐会拖慢调查进度。务必在日志里同时写入 `trace_id`、`frame_path`、`timestamp`，否则回放时只能靠模糊对照，运营判断会失真。

13.3 本章小结

工程监控的重点是把 metrics、visual evidence、回放逻辑合并为一个可检索的三链路，让事件发生时可以迅速定位到对应的 trace frame、judge dialog 和 solver plan，从而对故障做出精准响应。

14 章节回顾与反思

Lecture 05 的前 15 分钟通过计算成本与性能曲线设定了 planning domain 的边界：LLM 的 “smooth curve” 已经显示出 scaling 的边际收益递减，必须寻找更有结构性的解决路径。

Search Former / DU Forer 把 search trace 变成有 supervision 的序列，并用 dropout 模拟现实中不完备的搜索信息。Soang 和 Maze 的 benchmark 展示了 trace 模型在 generalization 上能以小模型击败 solution-only baseline，并为 multi-turn agent 提供更一致的建议。

多轮交互部分通过 APAC Constitution 与 agent judge 的 DPO loop 形成自监督闭环，强调四个维度的测量与 persona 差异化问句策略，让 agent 能在有限轮数内输出结构化 plan，显著降低人工对齐频率。

DSPy 框架把 latent cost C 作桥梁，串联 search trace、APAC 输出与 solver，从 combinatorial constraint 的难题中降维出可微的优化轨迹。Nano optics benchmark 证明了这套 pipeline 可以同时兼顾性能与工艺合法性。

这堂课的亮点在于把 search/agent/solver 串联为一条信息流，而非独立模块：每个输出都包含 time-stamped trace、judge reward、solver plan，构成完整的 audit log。

14.1 本章小结

回顾本节可以发现：从 search trace 到 multi-turn agent，再到 DSPy latent solver，整套架构围绕可解释性、结构化监督与工程可观测性展开，构成一个可回放、可调试的闭环。

15 总结与延伸

本讲通过 Compound AI 的三步路径：1) 捕获并压缩搜索轨迹，2) 用 APAC 宪法治理多轮问答，3) 通过 DSPy 学习复杂约束，在 1 小时内完整展示了从建模到工程落地的闭环。

这三部分之间不是孤立的：trace 模型用来指导 multi-turn agent 的问答流程，APAC 生成的结构化 Json 又是 DSPy 里 latent solver 的约束输入，最终的行动计划被送回 search trace 进行一致性检查，形成一个可审计的控制环。

主题	关键机制	教学价值
搜索轨迹	Trace-aware Transformer + DU Forer drop	提高 data/parameter efficiency、增强可解释性
多轮交互	APAC Constitution + agent judge/DPO	多轮提问具备 accuracy/proactivity/-credibility
复杂约束	Latent cost $C(Y)$ + solver + green descent	在非线性场景中嵌入可微反馈、兼顾制造约束

表 6: Compound AI 三步法的核心组件与价值

15.1 研究问题与扩展

AI 教练层的版本管理 当 search trace、APAC agent、DSPy solver 同时迭代时，容易出现“trace 参数与 agent 判断不同步”。建议把 trace ID、agent judge 模型版本、solver configuration 一起打包成 release 记录，在每次部署前跑自动化 test，确保新版 judge 仍能识别旧 trace，避免 unstable inference。

跨域泛化与 judge 校准 当前的 agent judge 在 travel intent 任务上表现突出，但在医疗、金融等高风险领域可能对 goal 的理解截然不同。可行的扩展策略是训练 judge 的 mixture-of-experts 或引入 domain embedding 让 judge 自适应 new intent，同时把 judge reward 与 human satisfaction signal (NPS、回访率) 对齐，而不是仅依赖 plan optimality。

多模态融合与辅助材料 讲者提到“slides、notebooks、photos”是加强解释的利器。理想情况下，search trace 应该能够引用 slides 中的表格/公式（如 Soang 的 sequencing graph），当视频里出现关键 diagram 时自动截帧并插入 note，保持视觉与文本双重证据。

- 如何进一步压缩 trace 的 token 数量以降低推理成本，同时保持多轮可解释性？
- agent judge 的泛化能力如何迁移到医疗、金融等高风险领域，而不仅仅是在 travel intent 里？
- 微分化的制造约束能否自动抽象成 latent representation，从而在新行业（例如芯片设计）间无缝迁移？

除了理论问题，这套 Compound AI 也要在 production 时持续沉淀：保持 trace/plan 的 audit log、把 DPO reward 与用户满意度对齐，并在不同 region 迭代 judge 模型，才能确保系统的长期健壮。

15.2 拓展阅读

- “Beyond A*: Better Planning with Transformers via Search Dynamics Bootstrapping”，用于理解搜索轨迹如何提升规划性能。

- “Dualformer: Controllable Fast and Slow Thinking by Learning with Randomized Reasoning Traces”, 对应 trace drop 与可控推理速度。
- “Composing Global Optimizers to Reasoning Tasks via Algebraic Objects in Neural Nets”, 讨论神经网络中的组合式全局优化。
- “SurCo: Learning Linear Surrogates For Combinatorial Nonlinear Optimization Problems”, 展示组合优化问题中的 surrogate 学习路径。
- APAC Constitution (2024), 内部白皮书, 详细定义了 Accuracy/Proactivity/Adaptivity/Credibility 的评价方法。

15.3 本章小结

Compound AI 的三步法不是孤立的实验, 而是面向 production 的组合: 把 visual slides、time-stamped frames、trace logs 融合进同一条 pipeline 里, 让观众不仅理解方法原理, 还能追踪到具体的工程实现。